DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-3

ASSIGNMENT 3 – CO3

CO3 Coverage: Context-Free Grammars for English syntax; Context-free rules and trees; Sentence-level constructions; Agreement; Subcategorization; Parsing (Top-down, Earley); Feature structures; Probabilistic Context-Free Grammars

Name: P. Prabhakar
Reg no: 192472134

Assignment Title: Design, Parse and Probabilistically Rank English Queries for a University Helpdesk Using Context-Free Grammars, Feature Structures and Earley Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Syntactic Analysis for a University Helpdesk Assistant

A university is building a text-based helpdesk assistant that answers student and staff queries about timetables, examinations, hostels, fees, library services and placements. Before any meaning can be extracted, every query must be assigned a reliable syntactic structure. The NLP team has decided to build a grammar-based parser for a controlled but realistic fragment of English, to resolve ambiguity with a probabilistic model, and to compare parsing algorithms on accuracy and cost.

You are the language-engineering team. You must investigate how English syntax can be captured by context-free rules, how agreement and subcategorization constrain those rules, how parsing algorithms search for trees, and how probabilities select the best tree when a query is ambiguous.

Planning data and constraints

- Grammar fragment: at least 40 phrase-structure rules and a lexicon of at least 60 words covering the helpdesk domain.
- Sample query set (build and document your own): at least 30 sentences, including declaratives, imperatives, yes/no questions, wh-questions and at

  least 5 structurally ambiguous sentences (e.g., “Show the students with the late fee notice”, “Book the room near the library for the seminar”).
- Required phenomena: subject–verb agreement, determiner–noun number agreement, pronoun case, and at least five verb subcategorization frames.
- The grammar must be written in Chomsky-style CFG notation; for the Earley and probabilistic tasks the grammar may be converted to a binarised or CNF-equivalent form where needed, and the conversion must be shown.
- Treebank probabilities: a small hand-annotated corpus of at least 20 parsed sentences may be used to estimate rule probabilities (maximum-likelihood estimates with smoothing explained).

- Any manual effort (grammar writing, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The helpdesk is to be extended to accept noisy, conversational and multi-clause student messages (e.g., "Can you tell me when the exams that were postponed will be held?"). Reevaluate your original grammar and parser and decide whether to retain, modify or redesign them.

TASK 1 – Design a Context-Free Grammar for English Syntax 5 Marks Guiding Enquiry

How can the core structure of English be captured by a finite set of context-free rules, and what design decisions make the grammar both expressive and manageable?

Investigate and interpret:

- Define the formal CFG **G = (N, Σ, R, S)** for your helpdesk fragment and state clearly the non-terminals, terminals (lexicon), rules and start symbol.
- Write rules for the major constituents: sentence (S), noun phrase (NP), verb phrase (VP), prepositional phrase (PP), adjective phrase, and coordination. Include recursive rules (e.g., NP → NP PP) and explain how they permit unbounded structure.
- Separate the grammar proper from the lexicon; list the parts of speech used and justify the granularity you selected.
- Explain the notions of constituency, head, and phrasal category with tests (substitution, movement, coordination) applied to at least three of your sentences.
- Show that your grammar is context-free by construction and discuss one English phenomenon that strains the context-free assumption.

Evidence Expected

A complete, well-organised CFG with lexicon, justified categories and worked constituency tests.

TASK 2 – Context-Free Rules, Derivations and Parse Trees 3 Marks Guiding Enquiry

How are sentences generated from the grammar, and how are trees used to represent and compare syntactic analyses?

Investigate and interpret:

- For five sentences from your query set, give a leftmost derivation and a rightmost derivation, and draw the corresponding parse tree.
- Represent each tree in both bracketed notation and tree diagram form, and compute its height, number of nodes and number of rule applications.
- Demonstrate ambiguity: for at least two sentences, give two distinct parse trees and describe the difference (e.g., PP-attachment, coordination scope).
- Explain the relationship between derivations and trees, and why many derivations may correspond to one tree.
- State a procedure to test whether a given string belongs to L(G) and discuss its limits.

Evidence Expected

Correct derivations, accurately drawn trees, and a clear analysis of structural ambiguity.

TASK 3 – Sentence-Level Constructions 4 Marks Guiding Enquiry

Which sentence types must the helpdesk understand, and how does the grammar represent each of them?

Investigate and interpret:

- Provide rules for the four main sentence-level constructions: declaratives, imperatives, yes/no questions and wh-questions (subject and non-subject).
- Handle auxiliary inversion, wh-movement (represented through a gap or an additional rule), and subordinate and relative clauses (e.g., "the exams that were postponed").
- Show parse trees for at least one sentence of each construction type and note the structural differences from the declarative.
- Discuss how coordination ("fees and hostel charges") and long-distance dependencies are treated and where the plain CFG becomes unwieldy.
- Analyse the effect of each new construction on grammar size and ambiguity.

Evidence Expected

Rules and trees for each sentence type, with an analysis of how the added constructions affect the grammar.

TASK 4 – Agreement 3 Marks Guiding Enquiry

How can a grammar prevent sentences such as "*The students submits the form" without producing an enormous set of rules?

Investigate and interpret:

- Identify the agreement phenomena in your fragment: subject–verb (number, person), determiner–noun, and pronoun case.
- First, enforce agreement by multiplying non-terminals (e.g., S → NP-sg VP-sg | NP-pl VP-pl). Count how many rules this requires for your grammar and explain the growth.
- Show how the rule explosion worsens as person, number, case and gender are combined, and compute the number of rules needed in the worst case.
- Propose a more economical alternative (feature-based rules) and show informally how it prevents ungrammatical strings; the formal treatment follows in Task 8.
- Test the original and agreement-aware grammars on at least six grammatical and six ungrammatical sentences and tabulate the results.

Evidence Expected

A quantified comparison of the baseline and agreement-aware grammars with test results.

TASK 5 – Subcategorization 4 Marks Guiding Enquiry

How does the grammar know which complements a verb may take, and how can this knowledge be encoded?

Investigate and interpret:

- Define subcategorization (valence) and list at least five frames for helpdesk verbs, e.g., intransitive ("The portal crashed"), transitive ("Submit the form"), ditransitive ("Send me the receipt"), sentential complement ("Tell me that the fee is paid"), and PP complement ("Apply for a hostel room").
- Encode the frames first with distinct verb categories (V-intr, V-trans, V-ditrans, Vcomp, …) and then explain the drawbacks of this approach.
- Distinguish complements from adjuncts using tests, with examples.
- Demonstrate how the grammar rejects over- and under-complemented sentences ("*Submit", "*The portal crashed the form") and show this in the parse results.
- Discuss verbs with multiple frames and how this changes ambiguity and parsing cost.

Evidence Expected

A subcategorization lexicon, grammar rules enforcing it, and tested accept/reject examples.

TASK 6 – Top-Down Parsing 4 Marks Guiding Enquiry

How does a top-down parser search for a tree rooted in S, and what are the strengths and weaknesses of this strategy?

Investigate and interpret:

- Describe the top-down (recursive-descent, goal-driven) algorithm in pseudocode, including how it expands non-terminals, matches words and backtracks.
- Trace the parser step by step on one unambiguous and one ambiguous sentence, showing the stack/agenda, the rule chosen and each backtracking point.
- Explain the problems of left recursion (e.g., NP → NP PP) and repeated re-parsing of the same subtrees, and show these problems on a concrete example.
- Show how left-recursion elimination or a bottom-up filter (left-corner) addresses these issues and discuss the effect on the resulting trees.
- Analyse the worst-case time behaviour and the number of redundant sub-parses for a sentence with k prepositional phrases.

Evidence Expected

Pseudocode, detailed traces and a correct analysis of the weaknesses of top-down parsing.

TASK 7 – Earley Parsing 5 Marks Guiding Enquiry

How does the Earley algorithm use dynamic programming to avoid the failures of simple top-down parsing, and what does its chart contain?

Investigate and interpret:

- Define an Earley state (dotted rule with start position) and explain the three operations: PREDICTOR, SCANNER and COMPLETER.
- Hand-simulate the algorithm on a helpdesk sentence of at least six words; show every chart entry (S[0]…S[n]) with its state, origin and the operation that created it.
- Recover the parse tree(s) from the chart by adding back-pointers and show how multiple trees are packed for an ambiguous sentence.

- Show how the algorithm handles a left-recursive rule and an ε-free ambiguous attachment without looping.
- Derive the complexity ($O(n^3)$ time, $O(n^2)$ space for the chart) and compare it with the top-down parser of Task 6 on the same sentence in terms of number of states generated.

Evidence Expected

A fully traced Earley chart, back-pointer tree recovery, and a justified complexity comparison.

TASK 8 – Feature Structures and Unification 4 Marks Guiding Enquiry

How can attribute–value structures and unification replace multiplied categories to handle agreement and subcategorization elegantly?

Investigate and interpret:

- Define feature structures (attribute–value matrices), feature paths, reentrancy (structure sharing) and represent at least five lexical entries with features such as NUM, PER, CASE and SUBCAT.
- Define unification and subsumption, and work through at least three unifications, including one that fails.
- Augment at least six grammar rules with feature constraints (e.g., S → NP VP with ⟨ NP AGREEMENT⟩ = ⟨ VP AGREEMENT⟩) and show the unified feature structure for one complete parse.
- Redo Task 4 (agreement) and Task 5 (subcategorization) using features and compare the grammar size with the earlier non-feature version.
- Describe how unification can be integrated with the Earley parser (unification during the COMPLETER step) and what is lost in complexity.

Evidence Expected

Correct feature structures and unifications, a feature-based grammar fragment, and a comparison with the non-feature grammar.

TASK 9 – Probabilistic Context-Free Grammars 5 Marks Guiding Enquiry

When a query has several valid parses, how can probabilities select the most plausible one?

Investigate and interpret:

- Define a PCFG: each rule $A \rightarrow \beta$ has a probability $P(A \rightarrow \beta \mid A)$, with the probabilities for each left-hand side summing to 1. Show that your grammar satisfies this condition.
- Estimate rule probabilities by maximum likelihood from your 20-sentence minitreebank, giving the counts, the formula and the resulting table; explain how you handle unseen rules.
- Compute the probability of at least two parse trees for an ambiguous sentence (e.g., PPattachment) and choose the best parse; compute the probability of the sentence as the sum over its trees.
- Describe the probabilistic CKY (Viterbi) algorithm in pseudocode after conversion to CNF, and compute the table for a short sentence.

- Discuss the independence assumptions of PCFGs (context-freeness, position invariance), the preference for shorter trees, and the lack of lexical sensitivity, illustrating each with a failure case.

Evidence Expected

A well-formed PCFG with estimated probabilities, worked tree probability calculations, a Viterbi trace and a reasoned discussion of limitations.

TASK 10 – Implement and Test the Parsers 3 Marks Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python (plain code or NLTK) the grammar, a top-down parser, an Earley parser and a Viterbi/PCFG parser, with feature checks for agreement and subcategorization.
- Run all parsers on the 30-sentence test set and report, for each: parse found or not, number of parses, number of chart states/steps and run-time.
- Include test cases for left recursion, ambiguous attachment, ungrammatical sentences and long sentences (up to 20 words), and a reference check against your hand-computed results from Tasks 7 and 9.
- Present the results in comparable tables and at least one plot (e.g., number of states against sentence length).

Evidence Expected

Working, documented code, a reproducible test procedure and results tables/graphs.

TASK 11 – Compare, Recommend and Defend the Parsing Design 2 Marks Guiding Enquiry

Which grammar-and-parser combination would you deploy in the helpdesk, and what evidence supports your choice?

Investigate and interpret:

- Compare the top-down, Earley and probabilistic parsers on coverage, ambiguity handling, cost and ease of grammar maintenance, using your analysis and measured results.
- Recommend a design (e.g., feature-based CFG + Earley + PCFG ranking), stating its expected accuracy and limitations.
- Discuss manual effort (grammar writing and annotation) and the point at which a statistical approach becomes more economical than hand-written rules.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the fastest option alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks Changed Requirement

The helpdesk must now accept conversational, multi-clause and partly ungrammatical student messages (for example, “Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?”). Messages may contain coordinated clauses, embedded questions, relative clauses and missing punctuation. At least 20% of the incoming messages are expected to contain a construction not covered by your grammar.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the constructions in the new messages that your grammar cannot parse (embedded questions, multi-clause coordination, ellipsis, fragments).
- Identify the affected assumptions: controlled vocabulary, fully grammatical input, small ambiguity, hand-written coverage.
- Estimate how the longer sentences affect the number of parses and the parsing time.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks Guiding Enquiry

How would your recommended parser behave on the new input?

- Run or hand-trace at least three new messages through your original grammar and parsers and record what fails first.
- Estimate the growth of ambiguity and chart size with sentence length using your Task 7 and Task 10 results.
- Identify whether another parsing strategy handles the new input more economically.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks Guiding Enquiry

Should the grammar and parser be extended, replaced, or supplemented?

- Choose retain, modify or redesign (e.g., add rules for embedded clauses, a robust/partial-parsing fallback, feature-based gap threading, a larger PCFG estimated from more data).
- Describe the revised grammar and parser and what changes in rules, features and probabilities.
- Determine the effect on coverage, ambiguity, parsing cost and annotation effort.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and quantified effect.

D. Evidence-Based Defence – 2 Marks Guiding Enquiry

Why is your final decision the right one given the evidence you generated?

- Use your derivations, traces, simulation results, coverage figures and effort estimates.
- Show that the decision follows logically from the evidence and state its remaining limitations.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Design a Context-Free Grammar for English Syntax	5
Task 2 – Context-Free Rules, Derivations and Parse Trees	3
Task 3 – Sentence-Level Constructions	4
Task 4 – Agreement	3
Task 5 – Subcategorization	4
Task 6 – Top-Down Parsing	4
Task 7 – Earley Parsing	5
Task 8 – Feature Structures and Unification	4
Task 9 – Probabilistic Context-Free Grammars	5
Task 10 – Implement and Test the Parsers	3
Task 11 – Compare, Recommend and Defend the Parsing Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

TASK 1 – DESIGN A CONTEXT-FREE GRAMMAR FOR ENGLISH SYNTAX (5 MARKS)

1.1 Formal CFG

A context-free grammar is G = (N, Σ, R, S), where N is a finite set of non-terminals, Σ is the set of terminals, R is a finite set of productions and S is the start symbol. For this grammar, S is the start symbol; N contains sentence, phrase and part-of-speech categories; Σ is the helpdesk lexicon; and R contains the rules below. Every production has exactly one non-terminal on its left-hand side.

1.2 Phrase-structure rules (48)

No.	Production
1	S → DECL
2	S → IMP
3	S → YNQ
4	S → WHQ
5	S → S CONJ S
6	DECL → NP VP
7	DECL → NP AUX VP
8	IMP → VP
9	YNQ → AUX NP VP
10	WHQ → WH NP VP
11	WHQ → WH AUX NP VP
12	WHQ → WH VP
13	NP → PRON
14	NP → N
15	NP → DET N
16	NP → DET ADJ N
17	NP → DET N PP
18	NP → NP PP
19	NP → NP REL
20	NP → NP CONJ NP
21	NP → POSS N
22	NP → DET ADJ ADJ N
23	VP → V_INTR
24	VP → V_TRANS NP
25	VP → V_DITR NP NP
26	VP → V_SENT CP
27	VP → V_PP PP
28	VP → AUX VP
29	VP → V_COP AP
30	VP → V_COP NP
31	VP → VP PP
32	VP → VP ADV
33	VP → V_TRANS NP PP
34	VP → V_TRANS CP
35	VP → VP CONJ VP
36	VP → V_TRANS NP ADV
37	PP → P NP
38	AP → ADJ
39	AP → ADV ADJ
40	CP → COMP DECL
41	CP → COMP WHQ

42	REL → RELPRO VP
43	REL → RELPRO AUX VP
44	AUXP → AUX VP
45	NP → DET N CONJ N
46	VP → V_TRANS NP PP
47	PP → P NP PP
48	VP → V_COP ADV AP

Recursive rules such as NP → NP PP, NP → NP REL and VP → VP PP permit phrases to contain further phrases. The grammar is finite, but recursion allows it to generate structures of unbounded length. The grammar is context-free because each rule rewrites one non-terminal without inspecting surrounding symbols.

1.3 Lexicon (60+ words)

Category	Lexicon
DET	the, a, an, this, that, these, those, my, your
PRON	I, we, you, he, she, they, it, me, us, them
POSS	my, your, his, her, our, their
N – people	student, students, staff, teacher, professor, applicant, librarian, officer
N – academic	exam, exams, timetable, schedule, result, grade, course, class, assignment, semester, department
N – services	hostel, room, library, book, portal, form, fee, fees, receipt, notice, account, scholarship, placement, interview, company, campus, laboratory, certificate
V_INTR	crash, crashes, arrive, arrives, expire, expires, open, opens, close, closes
V_TRANS	check, checks, submit, submits, show, shows, book, books, update, updates, download, downloads, need, needs
V_DITR	send, sends, give, gives, email, emails
V_SENT	tell, tells, know, knows, confirm, confirms, explain, explains, ask, asks
V_PP	apply, applies, wait, waits, pay, pays, register, registers
V_COP	be, is, are, was, were, become, becomes
AUX	do, does, did, can, could, will, would, should, may, might, is, are, was, were
P	in, on, at, for, from, to, with, near, before, after, during, about, of, by
ADJ	late, new, available, postponed, current, final, internal, online, overdue, academic, vacant, paid
ADV	quickly, recently, today, tomorrow, now, already, still
WH	what, when, where, why, how, which, who, whom
RELPRO	who, which, that
COMP	that, whether, if
CONJ	and, or, but

The grammar and lexicon are separate: grammar rules describe combination, while lexical entries identify category membership. Some words have more than one category. “That”, for example, may function as a determiner, complementiser or relative marker.

1.4 Constituency, heads and tests

A constituent is a group of words that behaves as a unit. Its head determines its category and important grammatical properties. In “the timetable”, timetable is the noun head of an NP; in “checks the timetable”, checks is the verb head of a VP; in “near the library”, near is the preposition head of a PP.

Sentence	Constituent	Test
The student checks the timetable.	the timetable (NP)	Substitution: “The student checks it.” The pronoun replaces the whole NP.
The librarian sent the receipt to me.	the receipt (NP)	Movement: “The receipt, the librarian sent to me.” The sequence moves as a unit.

The new exam and the hostel room are available.	the new exam / the hostel room	Coordination: two NPs combine in the same syntactic position.

Substitution, movement and coordination provide evidence for constituent boundaries, although no single test is decisive in every case. A phenomenon that strains a plain CFG is long-distance agreement or a dependency between a moved wh-word and a gap inside an embedded clause. Feature passing or a richer grammar formalism is more economical than duplicating many CFG rules.

TASK 2 – CONTEXT-FREE RULES, DERIVATIONS AND PARSE TREES (3 MARKS)

2.1 Five sentence derivations

Sentence	Rule sequence / derivation outline
The student checks the timetable.	S ⇒ DECL ⇒ NP VP; NP ⇒ DET N; VP ⇒ V_TRANS NP; object NP ⇒ DET N.
The librarian sends me the receipt.	S ⇒ DECL ⇒ NP VP; VP ⇒ V_DITR NP NP; subject NP ⇒ DET N; objects ⇒ PRON and DET N.
The portal crashes.	S ⇒ DECL ⇒ NP VP; NP ⇒ DET N; VP ⇒ V_INTR.
Show the students.	S ⇒ IMP ⇒ VP; VP ⇒ V_TRANS NP; NP ⇒ DET N.
When does the library close?	S ⇒ WHQ ⇒ WH AUX NP VP; NP ⇒ DET N; VP ⇒ V_INTR.

Leftmost derivation for “The student checks the timetable” (always expand the leftmost non-terminal):

S ⇒ DECL ⇒ NP VP ⇒ DET N VP ⇒ the N VP ⇒ the student VP ⇒ the student V_TRANS NP ⇒ the student checks NP ⇒ the student checks DET N ⇒ the student checks the N ⇒ the student checks the timetable

Rightmost derivation (always expand the rightmost non-terminal):

S ⇒ DECL ⇒ NP VP ⇒ NP V_TRANS NP ⇒ NP V_TRANS DET N ⇒ NP V_TRANS DET timetable ⇒ NP V_TRANS the timetable ⇒ DET N checks the timetable ⇒ the student checks the timetable

2.2 Bracketed tree and measurements

[S [DECL [NP [DET The] [N student]] [VP [V_TRANS checks] [NP [DET the] [N timetable]]]]]

Measure	Value	Convention
Terminal leaves	5	The, student, checks, the, timetable.
Non-terminal nodes	10	S, DECL, two NP, two DET, two N, VP and V_TRANS.
Total nodes	15	10 non-terminals plus 5 terminals.
Tree height	4 edges	Longest path from S to a lexical terminal.
Rule applications	10	One expansion for each non-terminal node.

A derivation is the sequence of rule applications; a tree records the resulting hierarchy. Different expansion orders may yield the same tree. Ambiguity exists only when one string has two structurally distinct trees.

2.3 Structural ambiguity

“Show the students with the late fee notice” has at least two readings. NP attachment: the students who have the notice. VP attachment: show the students by using the notice.

NP attachment: [S [IMP [VP show [NP [NP the students] [PP with [NP the late fee notice]]]]]]

VP attachment: [S [IMP [VP [VP show [NP the students]] [PP with [NP the late fee notice]]]]]

“Book the room near the library for the seminar” also permits PP attachment to the room or to the booking event; “for the seminar” may modify the room or the booking action. These different attachments produce different meanings.

2.4 Language membership

A string belongs to L(G) if S ⇒* w, where w consists only of terminals. A chart parser can test membership by checking whether a complete S spans the whole input. CFG membership does not guarantee semantic plausibility, and a restricted lexicon may reject valid English words that are not listed.

TASK 3 – SENTENCE-LEVEL CONSTRUCTIONS (4 MARKS)

Construction	Rule	Example
Declarative	DECL → NP VP	The student checks the timetable.
Imperative	IMP → VP	Show the timetable.
Yes/no question	YNQ → AUX NP VP	Does the library open today?
Wh-question – subject	WHQ → WH VP	Who checks the results?
Wh-question – non-subject	WHQ → WH AUX NP VP	When does the library close?
Subordinate clause	CP → COMP DECL / COMP WHQ	I know that the fee is paid.
Relative clause	NP → NP REL; REL → RELPRO VP	the exams that were postponed
Coordination	S → S CONJ S; NP → NP CONJ NP	Fees and hostel charges are due.

Declarative: [S [DECL [NP The student] [VP checks [NP the timetable]]]]
Imperative: [S [IMP [VP show [NP the timetable]]]]
Yes/no: [S [YNQ [AUX Does] [NP the library] [VP open today]]]
Wh-question: [S [WHQ [WH When] [AUX does] [NP the library] [VP close]]]
A yes/no question places an auxiliary before the subject. A non-subject wh-question fronts a wh-expression and uses auxiliary inversion. Subject wh-questions do not require the same inversion. Relative clauses modify an NP. Coordination combines phrases or clauses. Long-distance dependencies such as “What did the officer say that the student submitted __?” require a gap or feature-threading representation. Adding questions, relative clauses, embedded clauses and coordination increases grammar size and ambiguity.

TASK 4 – AGREEMENT (3 MARKS)

The required phenomena are subject–verb agreement (“The student submits” versus “The students submit”), determiner–noun number agreement (“this form” versus “these forms”), and pronoun case (“They emailed the officer” versus “The officer emailed them”).

Method	Approximate rule count	Explanation
Baseline	1 S → NP VP	Does not encode agreement by itself.
Number split	2 sentence rules	S → NP-SG VP-SG | NP-PL VP-PL.
Person × number	6 pairings	Three persons × two numbers.
Person × number × case × gender	36 feature bundles	3 × 2 × 2 × 3 combinations.
Naive subject/predicate crossproduct	Up to 1,296 pairings	36 subject bundles × 36 predicate bundles; many combinations are irrelevant.
Feature-based	1 schematic rule	S[AGR=x] → NP[AGR=x] VP[AGR=x].

Sentence	Baseline CFG	Agreement-aware	Reason
The student submits the form.	Accept	Accept	Singular agreement.
The students submit the forms.	Accept	Accept	Plural agreement.
They check the timetable.	Accept	Accept	Plural pronoun agreement.
She checks the result.	Accept	Accept	Third-person singular.
These forms are available.	Accept	Accept	Plural determiner and noun.
The officer sends me a receipt.	Accept	Accept	Correct subject and object.
The students submits the form.	May accept	Reject	Plural subject, singular verb.
These form are available.	Reject if DET number is encoded	Reject	Determiner–noun mismatch.
Her check the result.	May accept	Reject	Incorrect verb form.
Me checks the timetable.	May accept	Reject	Object case used as subject.
They checks the portal.	May accept	Reject	Plural subject, singular verb.
The student submit the form.	May accept	Reject	Third-person singular verb mismatch.

Feature unification uses shared variables: S[AGR=x] → NP[AGR=x] VP[AGR=x]. A subject with NUM=PL cannot unify with a VP requiring NUM=SG. This prevents ungrammatical combinations without multiplying rules.

TASK 5 – SUBCATEGORIZATION (4 MARKS)

Subcategorization (valence) is the lexical specification of the complements a verb requires. It determines the number and type of phrases that complete a verb.

Frame	Verbs	Pattern	Example
Intransitive	crash, arrive, expire	V_INTR	The portal crashed.
Transitive	submit, check, show	V_TRANS + NP	Submit the form.
Ditransitive	send, give, email	V_DITR + NP + NP	Send me the receipt.
Sentential complement	tell, know, confirm	V_SENT + CP	Tell me that the fee is paid.
PP complement	apply, wait	V_PP + PP	Apply for a hostel room.
Copular	be, become	V_COP + AP/NP	The room is available.

Rules: VP → V_INTR; VP → V_TRANS NP; VP → V_DITR NP NP; VP → V_SENT CP; VP → V_PP PP; VP → V_COP AP. Distinct verb categories encode frames clearly, but multiple-frame verbs require repeated lexical entries and the categories become cumbersome when tense, agreement and case are added. A complement is selected by the verb and completes its valence. An adjunct is optional information. In “The student submitted the form yesterday”, “the form” is the selected object and “yesterday” is an optional time adjunct. Optionality, selection restrictions and completeness tests help distinguish them.

Sentence	Result	Reason
The portal crashed.	Accept	Intransitive frame complete.
Submit the form.	Accept	Transitive object present.
Send me the receipt.	Accept	Two object complements present.
Tell me that the fee is paid.	Accept	Sentential complement present.

Apply for a hostel room.	Accept	Selected PP present.
Submit.	Reject for transitive frame	Required object missing.
The portal crashed the form.	Reject	Intransitive verb given an unlicensed object.
Send the receipt.	Reject in strict ditransitive frame	Recipient missing; another transitive frame could be separately licensed.

A verb may have more than one frame. “Book the room” is transitive, while “Book the room for the seminar” adds a purpose PP. Multiple frames improve coverage but increase ambiguity and parsing cost.

TASK 6 – TOP-DOWN PARSING (4 MARKS)

A top-down parser is goal-driven. It starts at S, expands non-terminals using grammar rules, matches terminals from left to right and backtracks when a selected rule fails.

6.1 Pseudocode parse(A, position): if A is a terminal: match input[position]; return next position or FAIL for each production A → X1 ... Xm: set current position to input position
parse each Xi from left to right, retaining possible end positions
if any Xi fails: backtrack and try another production if all Xi succeed: return the tree and end position return FAIL

6.2 Unambiguous trace: “The portal crashed.”

Step	Goal / stack	Action
1	S	Choose S → DECL.
2	DECL	Choose DECL → NP VP.
3	NP	Choose NP → DET N; match “The portal”.
4	VP	Choose VP → V_INTR; match “crashed”.
5	Input complete	Return one parse tree.

6.3 Ambiguous trace: “Book the room near the library.”

Attempt	Choice	Outcome
1	NP → NP PP	PP attaches to “the room”; room is near the library.
2	VP → VP PP	PP attaches to the booking event; booking occurs near the library.
3	Backtrack	Parser returns to the attachment point and explores the second alternative.

6.4 Left recursion and complexity

The rule NP → NP PP is left-recursive. A naive recursive-descent parser may call NP again at the same input position indefinitely. One rewrite is NP → NPB PP*, with NPB → DET N | DET ADJ N; PP* denotes zero or more PP modifiers. A left-corner parser is another solution. For k independently attachable PPs, a simplified NP/VP attachment model can have up to 2^k combinations. Naive parsing may be exponential in ambiguous cases; memoisation and chart parsing share repeated subproblems.

TASK 7- EARLEY PARSING (5 MARKS)

An Earley state is [A → α • β, i], where the dot shows how much of a production has been recognised and i is the origin position. Chart column S[j] contains states ending at input position j.

- PREDICTOR: when the dot precedes a non-terminal B, add B's productions at the current chart position.
- SCANNER: when the dot precedes a terminal matching the next word, advance the dot into the next chart column.
- COMPLETER: when a state is complete, find earlier states in its origin column waiting for its category and advance their dots.

7.1 Worked chart example

Input: "The student checks the new exam timetable." Token positions: 0 The, 1 student, 2 checks, 3 the, 4 new, 5 exam, 6 timetable, 7 end. For a compact demonstration, the object NP is binarised using auxiliary categories:

S → NP VP; NP → DET N; VP → V NP

NP → DET N1; N1 → ADJ N2; N2 → N N

Chart	Representative states and operation
S[0]	S' → • S,0 (seed); S → • NP VP,0 (PREDICTOR); NP → • DET N,0 (PREDICTOR).
S[1]	NP → DET • N,0 (SCANNER); N → • student,1 (PREDICTOR); lexical scan advances N.
S[2]	NP → DET N •,0 (COMPLETER); S → NP • VP,0 (COMPLETER); VP → • V NP,2 (PREDICTOR).
S[3]	VP → V • NP,2 (SCANNER); NP → • DET N,3 and NP → • DET N1,3 (PREDICTOR).
S[4]	Object NP scans DET "the"; NP → DET • N1,3. The next adjective category is predicted.
S[5]	N1 → ADJ • N2,4 after "new"; N2 → • N N,5 (PREDICTOR).
S[6]	N2 → N • N,5 after "exam"; second noun category is predicted.
S[7]	N2 → N N •,5; object NP completes; VP → V NP •,2; S → NP VP •,0; S' → S •,0.

The chart above gives the principal states for the illustrative binarised grammar; lexical preterminal states are represented by their scanner/predictor descriptions. A full implementation stores every lexical state explicitly. Completed states can retain back-pointers to the states and constituents that created them. Multiple back-pointers form a shared packed parse forest, allowing alternative PP attachments to share common subtrees.

Earley handles left recursion because duplicate states are inserted only once in a chart column; it does not recursively call itself indefinitely. For a fixed CFG, worst-case time is $O(n^3)$ and chart space is $O(n^2)$. Nearly unambiguous sentences are often faster in practice. Unlike naive top-down search, Earley shares states and avoids recomputing identical spans.

TASK 8 -FEATURE STRUCTURES AND UNIFICATION (4 MARKS)

A feature structure is an attribute–value representation. A feature path selects a nested attribute, such as AGR.NUM. Reentrancy means two paths share the same value. Unification combines compatible structures; conflicting values cause failure. Subsumption describes a general feature structure that is compatible with a more specific one.

Lexical item	Feature structure
student	[CAT=N, NUM=SG, PER=3]
students	[CAT=N, NUM=PL, PER=3]
she	[CAT=PRON, NUM=SG, PER=3, CASE=NOM]
them	[CAT=PRON, NUM=PL, PER=3, CASE=ACC]
submits	[CAT=V, NUM=SG, PER=3, SUBCAT=⟨NP⟩]
submit	[CAT=V, NUM=PL/base, SUBCAT=⟨NP⟩]
crashed	[CAT=V, TENSE=PAST, SUBCAT=⟨⟩]
send	[CAT=V, SUBCAT=⟨NP,NP⟩]

Unification	Result	Reason
[NUM=SG, PER=3] + [NUM=SG]	[NUM=SG, PER=3]	Compatible values.
[CASE=NOM, PER=3] + [CASE=NOM]	[CASE=NOM, PER=3]	Compatible values.
[NUM=PL] + [NUM=SG]	FAIL	Conflicting number values.

8.1 Feature-based rules

S[AGR=x] → NP[AGR=x] VP[AGR=x]

NP[NUM=n] → DET[NUM=n] N[NUM=n]

NP[CASE=NOM] → PRON[CASE=NOM]

VP[AGR=x] → V[AGR=x, SUBCAT=<>]

VP[AGR=x] → V[AGR=x, SUBCAT=<NP>] NP[CASE=ACC]

VP → V[SUBCAT=<NP,NP>] NP[CASE=ACC] NP[CASE=ACC]

VP → V[SUBCAT=<CP>] CP VP

→ V[SUBCAT=<PP>] PP

In “The student submits the form”, the subject has [NUM=SG, PER=3]. The VP must share that AGR value, selecting “submits”. The verb's SUBCAT=<NP> licenses the object. In “The students submits the form”, PL conflicts with SG and unification fails. Feature rules replace many split-category rules with shared constraints.

A feature-aware Earley parser carries feature structures in states and unifies a waiting state with a completed constituent during COMPLETER. Failed unifications are discarded. Feature operations add overhead and may enlarge the state space; the classic $O(n^3)$ bound assumes fixed-size grammar symbols.

TASK 9 -PROBABILISTIC CONTEXT-FREE GRAMMARS (5 MARKS)

A PCFG assigns P(A → β | A) to each production. Probabilities for all rules with the same left-hand side sum to 1. A tree probability is the product of its rule probabilities; a sentence probability is the sum of the probabilities of all its parse trees.

9.1 Twenty-sentence mini-treebank estimate

The following is an illustrative count summary from a hand-annotated 20-sentence teaching treebank. MLE is P(rule) = count(rule) / total count of rules with the same left-hand side.

Rule	Count	Calculation	P
S → DECL	14	14/20	0.70
S → IMP	2	2/20	0.10
S → YNQ	2	2/20	0.10
S → WHQ	2	2/20	0.10
DECL → NP VP	14	14/14	1.00
NP → DET N	12	12/16	0.75
NP → PRON	4	4/16	0.25
VP → V_INTR	4	4/14	0.286
VP → V_TRANS NP	8	8/14	0.571
VP → V_COP AP	2	2/14	0.143

For S, 0.70 + 0.10 + 0.10 + 0.10 = 1.00. For VP, 0.286 + 0.571 + 0.143 = 1.000 (rounding). For unseen rules, add-one smoothing can be used: P(A → β) = (count(A → β)+1)/(count(A)+K), where K is the number of candidate expansions.

9.2 PP attachment calculation

For “Book the room near the library”, suppose the shared lexical/lower-level factors contribute 0.25 to each tree, P(VP → V_TRANS NP)=0.60, P(NP → NP PP)=0.30 and P(VP → VP PP)=0.20.

Analysis	Simplified probability
NP attachment	0.25 × 0.60 × 0.30 = 0.045
VP attachment	0.25 × 0.60 × 0.20 = 0.030
Sentence probability	0.045 + 0.030 = 0.075
Preferred tree	NP attachment (higher probability).

These values are illustrative local scores. A complete PCFG computation multiplies every rule probability in each complete tree.

9.3 CNF and Viterbi CKY

CKY requires Chomsky Normal Form: rules are A → B C or A → terminal (with a possible special start rule). A long rule such as VP → V NP PP can be binarised by introducing X: VP → V X; X → NP PP. Unit and nullable productions must also be removed or handled.

> Initialise chart[i,i+1] with lexical categories and probabilities. For span length 2 to n, examine each start i and split k.
>
> For each binary rule A → B C, compute P(A → B C) × best(B,i,k) × best(C,k,j). Keep the maximum score and its back-pointer. Return best(S,0,n) and recover its tree.

For “students check forms”, lexical cells contain N, V and N. VP → V NP combines “check forms”; S → NP VP then combines “students” with that VP. The Viterbi score is the product of the binary-rule and lexical/preterminal probabilities. The numerical CKY table depends on the complete CNF probability table, so the recurrence is specified rather than inventing missing probabilities.

TASK 10 – IMPLEMENT AND TEST THE PARSERS (3 MARKS)

The proposed implementation uses Python and NLTK for CFG/Earley parsing, a recursive-descent demonstration for top-down parsing, and a CNF grammar with Viterbi CKY for PCFG ranking. Agreement and subcategorization are applied as feature validation constraints.

ID	Sentence	Type	Expected
Q01	The student checks the timetable.	Declarative	Accept
Q02	The students check the timetable.	Declarative	Accept
Q03	The librarian sends me the receipt.	Declarative	Accept
Q04	The portal crashed.	Declarative	Accept
Q05	The hostel room is available.	Declarative	Accept
Q06	The officer updated the account.	Declarative	Accept
Q07	Show the examination schedule.	Imperative	Accept
Q08	Submit the application form.	Imperative	Accept
Q09	Book the room near the library.	Ambiguous	Accept; 2 readings
Q10	Does the library open today?	Yes/no	Accept
Q11	Can the student download the result?	Yes/no	Accept
Q12	Will the hostel fee be due tomorrow?	Yes/no	Accept
Q13	When does the examination begin?	Wh-question	Accept
Q14	Where is the placement office?	Wh-question	Accept
Q15	Who checks the assignment?	Subject wh	Accept
Q16	Tell me that the fee is paid.	Sentential complement	Accept
Q17	I know whether the result is available.	Embedded clause	Accept
Q18	The exams that were postponed will begin soon.	Relative clause	Accept
Q19	Fees and hostel charges are due.	Coordination	Accept
Q20	The student checks the form with the receipt.	PP attachment	Accept; ambiguous
Q21	Show the students with the late fee notice.	PP attachment	Accept; ambiguous
Q22	Book the room near the library for the seminar.	Multiple PP	Accept; ambiguous
Q23	The student checks the new exam timetable today.	Longer declarative	Accept
Q24	The librarian emailed the officer yesterday.	Adjunct	Accept
Q25	The students submits the form.	Agreement error	Reject
Q26	These form are available.	Determiner error	Reject
Q27	Me checks the timetable.	Pronoun case error	Reject
Q28	The portal crashed the form.	Subcategorization error	Reject
Q29	Submit.	Missing complement	Reject in transitive frame
Q30	The student submit the form.	Agreement error	Reject

10.1 Reproducible test procedure

1. Install Python and NLTK: pip install nltk.
2. Save the grammar and parser program as helpdesk_parser.py.
3. Tokenise each query consistently and retain original text for display.

4. Run all 30 queries through top-down and Earley parsers; record parse success, parse count, chart states and elapsed time.
5. Convert the selected grammar to CNF and run Viterbi CKY.
6. Compare program output with manual derivations and the Task 7/9 examples.

Expected comparison: naive top-down parsing may revisit subproblems and backtrack heavily on ambiguous sentences; Earley stores chart states and supports left recursion; Viterbi CKY returns the highest-scoring parse. Exact runtime and state counts depend on machine, Python/NLTK version, grammar and implementation. No fabricated measurements are reported here.
Required plot: sentence length (tokens) on the x-axis and number of recursive calls/chart states on the yaxis, with separate series for top-down calls and Earley states. Populate the data by running the 30-query test set; do not draw a graph from invented measurements.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE PARSING DESIGN (2 MARKS)

Criterion	Top-down	Earley	PCFG / Viterbi
Coverage	Depends on rules; left recursion is problematic.	General CFG coverage.	Depends on CNF grammar coverage.
Ambiguity	Backtracking can enumerate trees.	Chart retains alternatives.	Ranks alternatives by probability.
Cost	Potentially exponential without memoisation.	$O(n^3)$ worst-case; $O(n^2)$ chart.	$O(n^3)$ for binary CKY.
Maintenance	Simple for small grammars.	Modular CFG rules.	Needs probability estimates and treebank updates.
Best role	Small deterministic patterns.	Main syntactic parser.	Disambiguation.

Recommendation: deploy a feature-based CFG parsed by Earley, followed by PCFG ranking. The feature layer checks agreement, pronoun case and verb subcategorization. Earley provides chart-based parsing and handles left recursion; PCFG ranking chooses a likely tree when alternatives remain. A partial-parser fallback handles fragments and unknown constructions.
Manual grammar writing is economical for a narrow, stable domain. As vocabulary and conversational patterns grow, hand-maintaining rules becomes expensive. A larger annotated corpus and statistical or neural models become more economical when sufficient representative data is available.

TASK 12 – DAY-4 DESIGN CHALLENGE (8 MARKS)

A. Impact analysis (2 marks)

The revised system must accept conversational, multi-clause and partly ungrammatical messages. Original assumptions of controlled vocabulary, fully grammatical input, limited ambiguity and complete handwritten coverage no longer hold. New constructions include greetings, discourse markers, embedded questions, coordinated clauses, relative clauses, ellipsis, fragments, missing punctuation and informal word order. Example: “Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?” This includes a greeting, a matrix request, embedded wh-clause, relative clause, passive future construction, coordination and an embedded whether-clause. The original compact grammar does not cover every part.
Longer sentences create more possible constituent boundaries and attachment sites. In highly ambiguous CFG fragments the number of trees can grow exponentially; Earley chart space remains $O(n^2)$ for a fixed grammar, with $O(n^3)$ worst-case processing.

B. Evaluation of original design (2 marks)

New message	Original design behaviour	Likely first failure
Hi, can you tell me when the exams that were postponed will be held?	May parse only if auxiliary, embedded wh, relative and passive rules are present.	Greeting and embedded wh/passive VP.
the library open today?	Grammar expects explicit auxiliary or a licensed declarative pattern.	Missing auxiliary / fragment.
I need hostel room and when fee pay?	Lexical words may exist, but ellipsis and malformed embedded question are not licensed.	Missing determiners, inflection and clause structure.

Earley can avoid repeated state expansion but cannot parse a construction absent from the grammar. A PCFG cannot rank a tree that the grammar never generates. Coverage is therefore a grammar/data problem as well as a parser problem.

C. Adaptation or redesign (2 marks)

Decision: MODIFY the original design. Retain the feature-based CFG for predictable domain syntax, expand it for embedded clauses and passive constructions, and add a robust partial-parsing fallback. Train a larger PCFG on representative annotated messages.

S → GREETING S | S DISCOURSE CONJ S | DECL | YNQ | WHQ

GREETING → hi | hello | good morning

CP → COMP WHQ | COMP DECL

WHQ → WH AUX NP VP | WH NP VP

VP → AUX VP | AUX BE V_PART | V_SENT NP CP

REL → RELPRO AUX VP

DECL → NP AUX VP | NP VP

FRAG → NP | PP | NP VP (partial-input fallback)

VP → VP ADV | VP PP

Component	Original	Modified	Expected effect
Grammar	Controlled phrase rules	Embedded, passive, discourse and fragment rules	Coverage rises; ambiguity also rises.
Parser	Top-down / Earley comparison	Feature-aware Earley primary	Shares states; supports left recursion.
Ranking	Small illustrative PCFG	Larger domain treebank PCFG	Better ranking if data is representative.
Fallback	None	Partial parse + unknownword handling	Graceful handling of incomplete messages.
Training data	20-sentence teaching treebank	Hundreds/thousands of annotated queries over time	Better estimates; more annotation effort.

D. Evidence-based defence (2 marks)

The decision follows from earlier evidence. Task 4 showed that unification checks agreement without multiplying rules. Task 5 showed that subcategorization rejects missing or extra complements. Task 6 demonstrated that naive top-down parsing can loop on left recursion and repeat work under ambiguity. Task 7 showed that Earley stores states by input position and handles left recursion through chart deduplication. Task 9 showed that PCFGs rank alternative trees, but their quality depends on the treebank. Task 10 provides a reproducible 30-query evaluation plan without inventing runtime values.

The final pipeline is: (1) normalisation and tokenisation; (2) feature-based CFG; (3) feature-aware Earley parsing; (4) PCFG ranking; (5) partial parsing when no complete parse exists; and (6) clarification when confidence is low. Limitations include finite domain coverage, annotation cost, sparse probabilities, incorrect parses for noisy input and the need for semantic interpretation after syntax.

Student message → normalisation/tokenisation → feature CFG → Earley chart → PCFG ranking
→ full parse / partial parse / clarification → helpdesk intent extraction

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Design a Context-Free Grammar for English Syntax	5
Task 2 – Context-Free Rules, Derivations and Parse Trees	3
Task 3 – Sentence-Level Constructions	4
Task 4 – Agreement	3
Task 5 – Subcategorization	4
Task 6 – Top-Down Parsing	4
Task 7 – Earley Parsing	4
Task 8 – Feature Structures and Unification	4
Task 9 – Probabilistic Context-Free Grammars	5
Task 10 – Implement and Test the Parsers	3
Task 11 – Compare, Recommend and Defend the Parsing Design	2
Subtotal	**41**
Task 12 – Day-4 Design Challenge	8
TOTAL	**49**